

THE DOSYAGO CORPORATION

FREELANG

THE CANONICAL GUIDE

INTRODUCTION, LANGUAGE, AND PRACTICE

DOCUMENT: FL-1
EDITION: SECOND PROVISIONAL EDITION, 2026
STATUS: CANONICAL PRE-RELEASE GUIDE
FORMAT: PRINTED REFERENCE

PREPARED FOR PROGRAMMERS, BUILDERS, AND MACHINE AUTHORS

THE FREELANG BUILDER'S KIT. Companion source, exercises, compiler materials, and working programs are available separately. For ordering information write freelang@dosaygo.com or compiler@freelang.dev.

Copyright © 2026 The Dosyago Corporation.

Freelang is pre-release software. This book is the self-contained canonical introduction to the current language. No outside document is required to understand its design position. Code, tests, and tagged release evidence remain authoritative when a pre-release implementation defect or recent change makes printed detail temporarily inaccurate.

The project is dual-licensed under AGPL-3.0-or-later or a commercial license. Third-party names are used only to identify compatible systems and prior art. No warranty is expressed or implied.

No publication metadata is retained in the distributed PDF. This notice is part of the printed work.

PREFACE

Freelang is a small ahead-of-time systems language intended for a time when machines will write far more programs than people have written before. Its premise is simple: producing code faster does not excuse reproducing familiar mistakes faster. The language therefore tries to make authority explicit, world failure enumerable, programmer error fatal, concurrency isolated, and the route from source to native artifact short enough to inspect.

The compiler is ordinary JavaScript. The programs it produces are not. They are direct ELF, Mach-O, PE32+, or WebAssembly artifacts, with a compact runtime and no general in-process foreign-function interface. Linux, macOS, and Windows are current native targets. Stateful foreign capability, when required, is placed behind a separate process and a narrow protocol.

This guide is deliberately conventional in arrangement. It begins with a one-line program, proceeds through values and control, and ends with systems, proofs, and substantial examples. That arrangement should not be mistaken for a conventional thesis. Freelang asks whether a language can be both pleasant to generate and severe about what generated programs are permitted to leave ambiguous.

Examples in this book are drawn from the repository and from the companion Builder's Kit. They should be typed, compiled, altered, and inspected. Reading alone is insufficient preparation for programming.

CONTENTS

Preface	iii
1 The Purpose of the Language	1
1.1 The problem being addressed	1
1.2 Safe amplification	1
1.3 The design center	2
1.4 Explicitness and the test for magic	2
1.5 Bugs, world states, and refusal	3
1.6 Costs are part of the meaning	3
1.7 Ten governing rules	4
1.8 Proof is part of the product	4
2 Lineage and Departures	5
2.1 Assembly and C	5
2.2 JavaScript and TypeScript	5
2.3 Python and Ruby	5
2.4 Go and communicating processes	6
2.5 Rust	6
2.6 Erlang and Elixir	6
2.7 Ada, SPARK, and high-integrity practice	7
2.8 Unix, shells, and filesystem truth	7
2.9 Capability systems	7
2.10 The resulting synthesis	7
3 Obtaining and Running the Compiler	9
3.1 Two distributions	9
3.2 The first source build	9
3.3 Explicit compilation	9
3.4 What the compiler does	9
4 A First Program	11
4.1 Names and values	11
4.2 Interpolation	11
4.3 Comments	11
5 Integers, Logic, and Control	12
5.1 Arithmetic and comparison	12
5.2 Conditional blocks	12
5.3 Repetition	12
6 Arrays, Bytes, and Objects	13
6.1 Arrays	13
6.2 Strings and bytes are not the same thing	13

6.3	Objects and fields	14
7	Operations	15
7.1	Shaped parameters	15
7.2	Infix vocabulary	15
7.3	Private operations and references	15
8	Credits, Modules, and Authority	17
8.1	File-relative modules	17
8.2	Explicit scope	17
9	Bugs and the World	18
9.1	A bug ends the program	18
9.2	The world is data	18
9.3	Handling, retrying, and bubbling	19
10	Jobs and Isolation	20
10.1	The smallest fork and join	20
10.2	FSABI in one sentence	20
10.3	A complete value round trip	21
10.4	The capsule on disk	22
10.5	The lifecycle	22
10.6	The value format	23
10.7	Why the format is not JSON	24
10.8	Objects and operations cross by identity that can be checked	24
10.9	The trust boundary is the route, not the header	24
10.10	Copy semantics and refused values	25
10.11	Joining several jobs	25
10.12	Unix fork and Windows re-execution	25
10.13	Jobs, external programs, and sidecars are distinct	26
10.14	What FSABI does and does not prove	26
10.15	Cost, grain, and failure domain	26
11	Foreign Capability and Sidecars	27
11.1	The five properties	27
11.2	Chunky, never chatty	27
11.3	What the claim means	27
12	Files, Networks, and Windows	28
12.1	Text at boundaries	28
12.2	Network outcomes	28
12.3	A graphics loop	28
13	Memory and Native Artifacts	29
13.1	The value representation	29
13.2	Collection	29
13.3	Direct containers	29

14 WebAssembly Without Ambient JavaScript	30
14.1 The Doom application	30
15 A Tour of the Example Programs	31
15.1 Small transformations	31
15.2 Auditable system tools	31
15.3 Servers and interfaces	31
15.4 Substantial applications	32
16 A Disciplined Building Method	33
16.1 Begin with the world	33
16.2 Name authority	33
16.3 Keep invariants fatal	33
16.4 Build from applications, then retain proofs	33
16.5 Inspect what was built	33
17 The Position in Full	34
17.1 The language contract	34
17.2 The engineering contract	34
17.3 The admitted limits	34
17.4 Why the guide remains provisional	35
A Syntax Summary	36
B Compiler Command Summary	37
C Glossary	38
Builder’s Kit Notice	40

SECTION 1

THE PURPOSE OF THE LANGUAGE

1.1 The problem being addressed

Freelang begins from a change in the expected scale of programming. Machine authors can produce source faster than human organizations can inspect it. A language designed only to make an individual programmer productive may therefore optimize the wrong quantity. If a familiar mistake remains easy to express, a model can reproduce it thousands of times before the first instance has been understood.

The objective is not to make generated code respectable by declaration. It is to reduce the number of things an author must remember, make the remaining obligations visible in source, and arrange for violated invariants to stop near their cause. This is a language-design problem before it is a prompting or review problem. An instruction to “be careful” is not a structural guarantee.

1.2 Safe amplification

The useful question is not merely whether an individual program has a low defect rate. If software production increases by orders of magnitude, a recurring footgun can produce more total defects even when models make the mistake less often than people did. Freelang calls the desired alternative *safe amplification*: increase what people and models can build while structurally removing recurrent classes of error and keeping the remaining trusted surface finite.

Safe amplification changes the arithmetic of feature design. Suppose a footgun appears in one program out of ten thousand. That may have seemed tolerable when a team wrote hundreds of programs. It is not tolerable when machines can write millions. The language should remove the recurring mistake where it can, refuse it where it cannot assign honest semantics, and expose it as a named obligation where the world itself remains uncertain.

This is an objective, not a declaration of completion. It supplies four tests for a feature:

1. Can the feature create plausible but wrong output without a visible failure?
2. Does it introduce authority that cannot be identified at the source operation?
3. Does it enlarge the trusted in-process surface or merely compose an existing bounded mechanism?
4. Can the claim made for it be checked independently on every target?

A convenience that introduces ambient authority, hidden failure, or an unbounded native boundary therefore bears a heavier burden than a visible operation with a small contract.

1.3 The design center

Freelang was designed by one human design center in continuing collaboration with machine models. The human supplied the taste, refusals, syntax, questions, and final authority. Models generated substantial compiler and runtime work, proposed alternatives, found cross-platform consequences, and built much of the differential and stress evidence used to keep direct native code honest.

The result is neither anonymous machine output nor a traditional solitary compiler. It is an authored human–model collaboration whose intended users include both kinds of author. Human readers must be able to judge generated source. Machine authors must be able to hold enough of the semantic surface in context to reason about it without folklore.

The project began as an attempt to make a small ahead-of-time JavaScript. Its author had learned programming through 8086 machine instructions entered with DOS DEBUG, followed by QBasic, C, Perl, Java, and a long period of web and JavaScript work. Direct machine behavior and expressive construction were therefore present at the beginning. Compatibility with JavaScript ceased to be an objective once building a compiler made inherited language choices feel optional.

An early development loop asked models to translate design intent directly into assembly and executable containers. The compiler later acquired an IR, shared runtime generators, and independent binary writers, but that experience fixed an important point: source remains valuable as the durable statement of intent even when a machine can cheaply regenerate the artifact. The source must name authority, failure, and isolation clearly enough that regeneration does not become permission to forget them.

The concurrency design was sharpened by a practical failure. While building a concurrent terminal application in Go, capable models repeatedly entered well-known concurrency traps. Moving the immediate application work to Rust helped, but the larger lesson remained. Prestige, popularity, and model intelligence do not neutralize the cognitive shape of a language. If an error pattern is recurrent, its reproduction at machine scale matters more than the reputation of the feature that permits it.

Human authorship and model contribution have distinct roles. The human design center supplies taste, refusal, final judgment, and responsibility for the whole. Models have supplied large amounts of implementation, cross-platform analysis, adversarial criticism, and proof machinery. The resulting method is not “the model wrote it” and not solitary authorship assisted by an elaborate autocompleter. It is directed human–model construction with one identifiable human authority and a deliberately published evidence trail.

1.4 Explicitness and the test for magic

Freelang uses the word *magic* narrowly. Magic is mechanism that can change what a program computes without appearing in the source in a form the reader can identify. The practical test is a trace: when an important action occurs, can the reader point to the source construct that authorized it?

This does not reject abstraction. A garbage collector is hidden machinery, but its

acceptable operation is limited to reclaiming unreachable storage. It may affect when memory is reused; it must not alter a value observed by the program. Type coercion fails the same test because it can quietly turn a wrong type into a plausible wrong answer. A process fork is acceptable because the `&` is present. Joining is acceptable because `wait` is present. Module authority is acceptable because `credits:` is present.

The compact rule is:

If it matters, make it visible. If it is impossible, refuse it loudly.

Explicitness is not a claim that every machine instruction belongs in source. It is a demand that semantic events, authority changes, failure paths, and cross-process transfers have visible owners.

1.5 Bugs, world states, and refusal

Freelang divides unsuccessful computation into three categories.

Programmer bug

A bound is violated, a field is absent, a heap header is corrupt, a value is cyclic where the boundary forbids cycles, or an asserted invariant is false. The program falls. Recovery would let a lie travel.

World state

A file is absent, a peer closes, permission is denied, or a process cannot start. The operation declares a chaos domain and returns a named value from it. The caller chooses policy.

Unsupported claim

A target or boundary cannot honestly provide the requested behavior. The compiler or operation refuses it by name rather than approximating it.

The distinction is load-bearing. Turning a broken invariant into chaos would suggest that ordinary callers can proceed safely. Turning an expected network close into a fatal bug would deny reality. Pretending a reduced-authority Web target can provide native process jobs would make the platform claim false.

1.6 Costs are part of the meaning

Freelang does not advertise every safety choice as free. Process jobs are heavier than green threads. Explicit world handlers occupy source. A tracing collector adds runtime and trusted code. No general FFI means that foreign capability requires a protocol and a second executable. Direct backends mean the project must prove several calling conventions and container formats.

These costs are accepted where they purchase simpler invariants or a smaller failure domain. They remain subject to measurement. A costly mechanism that does not remove the promised ambiguity has failed twice.

1.7 Ten governing rules

The language is guided by the following rules.

1. Source shall remain legible to both human and machine authors.
2. Programmer bugs shall fail loudly and nearby.
3. External uncertainty shall be represented as explicit world state.
4. Concurrency shall name isolation rather than shared mutable memory.
5. Generated programs shall have a small, declared native surface.
6. Foreign capability shall cross a process and protocol boundary.
7. Readable source shall remain the behavioral specification.
8. Native artifacts shall carry inspectable evidence.
9. An escape hatch shall not grow into a second language.
10. Claims shall remain smaller than their proofs.

The short project motto is: *No more magic. Reality is magic enough.*

1.8 Proof is part of the product

The compiler's behavior suite, expected-failure programs, backend differentials, exact encoding probes, small-heap collection runs, sidecar drivers, release manifests, signatures, and artifact inspectors are not administrative decoration. They are part of the design. A direct native backend is valuable only if independent checks can distinguish an executable that implements the source contract from one that merely launches.

Proof remains bounded. Passing the current suite does not prove all programs. Avoiding libc does not establish memory safety. Process isolation does not make a protocol correct. A signed artifact establishes publisher identity and integrity, not good behavior. The project states the smaller claim and retains the evidence for that claim.

SECTION 2

LINEAGE AND DEPARTURES

Freelang is not presented as an invention without ancestors. Some features were consciously informed by earlier systems; others arose independently and later converged with known practice. The useful question is not who first used a brace, process, or capability. It is which inherited idea is retained, which assumption is rejected, and how the resulting composition differs.

2.1 Assembly and C

From assembly culture Freelang retains mechanical sympathy, direct artifacts, explicit layout work in the compiler, and the belief that a programmer should be able to inspect what runs. From C it shares ahead-of-time compilation, closed named operations, ordinary integer computation, and a preference for a short route to the machine.

It departs from C at the points where machine permissiveness becomes semantic ambiguity. Array and buffer bounds are checked. Heap values carry tags and sizes. Strings are codepoints rather than zero-terminated byte pointers. Generated programs do not expose a general pointer language, preprocessor, undefined-behavior contract, user-facing in-process FFI, or implicit conversion table. There is no general libc or CRT surface in the program's language boundary. Freelang spends a collector and runtime checks to avoid asking every author to reproduce C's discipline perfectly.

2.2 JavaScript and TypeScript

JavaScript contributed a taste for expressive construction, lightweight objects, first-class callable references, interpolation, and a small amount of syntax doing useful work. Freelang's implementation is itself ordinary JavaScript, which keeps the compiler accessible to the large population of programmers and models that can inspect that language.

Freelang is not a JavaScript runtime or a TypeScript ahead-of-time backend. There is no prototype chain, property getter, proxy, promise queue, implicit event loop, automatic semicolon insertion, truthy object convention, or coercive `+`. Object shapes are finite compiler-known layouts. A field read is a field read. Compilation produces a standalone native container or a reduced-authority WebAssembly module rather than shipping the source into a general JavaScript virtual machine.

2.3 Python and Ruby

Freelang shares their concern for readable source and rapid expression of ordinary programs. Its dynamic tagged values can feel closer to these languages than to a conventional static systems language.

The similarity ends before their ambient object models. Freelang has no descriptor protocol, `method_missing`, open class, executable import, exception unwinder, or

duck-typed missing-member discovery. Operations declare their positional or shaped inputs. Modules are credited explicitly. The cost is less metaprogramming and fewer convenient recovery shortcuts; the intended gain is that a source read does not require knowledge of distant runtime hooks.

2.4 Go and communicating processes

Go validates several premises Freelang also values: small source, native deployment, practical concurrency, explicit error handling, and a runtime that can support productive systems work. Go's goroutines and channels descend from CSP and make large numbers of communicating tasks economical.

Freelang chooses a different isolation unit. A job is an operating-system process with its own collector and heap, not a goroutine scheduled inside a shared address space. It has no shared slices, maps, pointers, scheduler, or channel network. `&` and `wait` show birth and join; FSABI records value transfer in an inspectable capsule. This is far heavier and deliberately unsuitable for millions of tiny tasks. It removes the possibility that a job races another by mutating the same heap object.

2.5 Rust

Rust is important to Freelang as proof that recurring memory hazards should be structurally excluded rather than documented as programmer obligations. Rust uses static ownership, borrowing, lifetimes, and explicit `unsafe` blocks to provide memory safety without a tracing collector.

Freelang makes a different trade. It uses dynamic tagged values and a small non-moving tracing collector. Its source has no lifetime calculus and no general `unsafe` mode. Process isolation addresses concurrent mutation by removing the shared heap rather than proving synchronized access to it. This is easier to generate and read, but pays in runtime work and gives the collector a place in the trusted base. Freelang does not presently claim Rust's static memory-safety guarantee. Linear resource handles may be useful in the future, but only where application evidence justifies them.

2.6 Erlang and Elixir

Erlang provides the nearest well-known validation of "let processes fail and communicate by values." Erlang ports also establish the value of placing foreign code in an external process instead of permitting it to corrupt a language runtime. Freelang's jobs and sidecars strongly resemble these two ideas at the level of isolation.

They differ in execution and emphasis. An Erlang process is lightweight VM state with a mailbox and supervision culture. A Freelang job is an OS process, uses explicit `fork/join` syntax, and transfers values through a filesystem capsule that can be inspected without the runtime. There is no general actor mailbox, selective receive, hot-code loading, or distributed VM. Freelang's sidecars add hash-pinned identity, release evidence, bounded lifecycle, and protocol drivers to the external-process

pattern.

2.7 Ada, SPARK, and high-integrity practice

Ada and SPARK demonstrate the value of explicit contracts, restricted subsets, bounded execution arguments, and evidence that is stronger than ordinary tests. Freelang shares the preference for named obligations and for profiles that may refuse convenient but unproved behavior.

Freelang is currently much smaller and does not claim SPARK's mature static proof system or certification history. Its evidence is presently a practical stack of executable tests, differential oracles, runtime invariant checks, protocol conformance, and artifact inspection. The inspiration is the discipline of making claims checkable, not a claim of equivalent assurance.

2.8 Unix, shells, and filesystem truth

The Unix tradition contributes processes, byte streams, files as durable interfaces, small tools, and the idea that external behavior can often be observed with ordinary system utilities. Fork syntax and output redirection in Freelang deliberately echo shell notation.

Freelang does not inherit shell word splitting, textual command composition, ambient environment inheritance, PATH search, or stringly typed pipelines. An external program receives an absolute executable path, exact argument array, and exact environment array. An FSABI value carries a version and type; arbitrary artifacts stay bytes. Familiar punctuation therefore names a stricter protocol than its shell ancestor.

2.9 Capability systems

Capability-oriented security supplies a final major convergence: authority is safer when it is explicit, narrow, and transferable only through controlled paths. Freelang's credits: declarations, intrinsic manifest, sidecar authority classes, and refusal of a general FFI all move in this direction.

Credits are not yet a complete object-capability security system. A credit declares and limits source-callable vocabulary; operating-system enforcement varies by boundary, and capability-bound replayable job capsules remain an area of continuing work. The difference between a declaration and an enforced sandbox is stated rather than hidden.

2.10 The resulting synthesis

Freelang can be summarized as JavaScript-like expressive construction over compiler-known shapes; C-like direct native production without C's source memory model; Erlang-like isolation made deliberately heavier and visible on disk; Rust-like structural

refusal without a borrow checker; and high-integrity-style proof obligations applied to a small model-oriented compiler.

That combination is distinctive. Distinctive does not mean unprecedented in every component, and it does not establish historical importance. The test is whether unfamiliar people and models can build unfamiliar programs, make fewer silent mistakes, and leave artifacts another party can understand.

SECTION 3

OBTAINING AND RUNNING THE COMPILER

3.1 Two distributions

Freelang may be used from source or as a standalone compiler. The source distribution requires Node 24 or later. The standalone distribution packages the compiler, its standard library, and its target-indexed sidecar material in one host executable. It does not require an installed Node, Python, assembler, linker, or native compiler toolchain for ordinary compilation.

The release archive contains `freelang` (or `freelang.exe`), a short `README`, and `BUILD-EVIDENCE.json`. Keep the evidence file with the release even though the compiler does not require it to run.

3.2 The first source build

From a source checkout on Linux or macOS:

```
| ./flx.sh examples/hello.flx
```

On Windows:

```
| .\flx.ps1 examples\hello.flx
```

The wrapper compiles for the host, runs the result, and passes arguments after the source name to the program. Use `-build-only` when execution is not wanted.

3.3 Explicit compilation

The standalone compiler and the source entry point accept the same essential shape:

```
| ./freelang input.flx output.bin --target=linux
| ./freelang input.flx output.bin --target=darwin
| ./freelang input.flx output.bin --target=darwin-arm64
| freelang.exe input.flx output.exe --target=windows
```

The four native target names produce direct executable containers. The `-emit=asm` option retains readable assembly on the x86-64 paths. A Web build uses `-target=wasm` and an appropriate Web emission mode.

A cross-target executable is built, not emulated. A Linux binary produced on macOS must be transferred to a Linux system before it can run.

3.4 What the compiler does

The path is intentionally short:

source $\longrightarrow$ frontend $\longrightarrow$ IR $\longrightarrow$ target backend $\longrightarrow$ executable container

The native backends emit ELF64, Mach-O, and PE32+ directly. LLVM is not in the production path. The runtime is emitted with the program and includes the allocator, collector, diagnostics, and only the credited or reachable facilities required by that program.

SECTION 4

A FIRST PROGRAM

Create a file named `hello.flx`:

```
| print "Hello world";
```

Compile and run it. The semicolon ends the statement. The quoted material is a Freelang string. The name `print` is a built-in observation operation and does not require a module credit.

4.1 Names and values

A binding is introduced with `let`.

```
| let greeting = "Hello";  
| let answer = 6 * 7;  
  
| print greeting;  
| print answer;
```

Bindings are lexical. An inner block may shadow an outer name. Values are integers or heap values: strings, arrays, bytes, objects, operation references, jobs, and closed state values. Integer zero is false; every nonzero integer is true.

4.2 Interpolation

The sequence `^{expression}` inserts a value into a string.

```
| let language = "Freelang";  
| let year = 2026;  
| print "^{language}, provisional edition ^{year}";
```

Interpolation is explicit source syntax. The outside world receives UTF-8. Inside the language, a string is a sequence of Unicode codepoints, so indexing a string selects a character rather than one byte of a multibyte encoding.

4.3 Comments

The sequence `//` begins a comment extending to the end of the line.

```
| // This is not emitted into the program.  
| print 42; // Nor is this.
```

SECTION 5

INTEGERS, LOGIC, AND CONTROL

5.1 Arithmetic and comparison

The ordinary integer operations are `+`, `-`, `*`, `/`, and `%`. Division is integer division. Comparisons produce zero or one.

```
let width = 80;
let height = 25;

print width * height;
print width > height;
print width == 80;
```

Boolean operators are words: `and`, `or`, and `not`. The first two short-circuit. They are deliberately distinct from bit operations, whose spellings are `*&`, `*|`, `*^`, `*!`, `*<`, and `*>`.

5.2 Conditional blocks

Freelang uses parentheses for both the condition and the controlled block.

```
let score = 85;

if (score >= 90) (
  print "A";
) else if (score >= 80) (
  print "B";
) else (
  print "C";
)
```

Conditions use the language's integer truth convention. Code should state the particular fact it intends to test instead of relying on an incidental value.

5.3 Repetition

```
let counter = 5;
while (counter > 0) (
  print counter;
  counter = counter - 1;
)
```

`done` leaves a loop. Labeled `done` name and `now` name provide explicit nonlocal control where ordinary nesting is insufficient.

SECTION 6

ARRAYS, BYTES, AND OBJECTS

6.1 Arrays

Arrays are ordered, checked, positional values.

```
let primes = [2, 3, 5, 7, 11];
print primes\0;

let i = 3;
print primes\\i;
print primes\\i;
```

The first read uses a literal slot. The latter two are coequal dynamic slot spellings. Every access is bounds-checked. An invalid slot is a programmer bug and ends the program; it does not fabricate a value.

Arrays are immutable after construction. Concatenation creates a new array.

```
let left = [1, 2];
let right = [3, 4];
let whole = left + right;
```

The forall form traverses every element of an array. In expression form it constructs a new array.

```
forall value in [10, 20, 30] (
  print value;
)

let squares = forall value in [1, 2, 3, 4] (
  => value * value;
);
```

6.2 Strings and bytes are not the same thing

A string contains codepoints. A byte array contains raw octets. The two are not silently confused at filesystem or network boundaries. Encode text with `string_to_utf8_bytes`; decode it with `utf8_to_string`.

```
credits: <f/bytes>;

let wire = string_to_utf8_bytes["café"];
let text = utf8_to_string[wire];
print text;
```

Byte literals begin with a percent sign:

```
let header = %[70, 76, 88, 1];
```

Typed scalar buffers use related visible forms where required by numeric and graphics work. They preserve their raw representation instead of pretending to be ordinary boxed arrays.

6.3 Objects and fields

Objects have named shapes.

```
let point = {x: 12, y: 30};
print point\x;

let field = \y;
print point\\field;
print point\@field;
```

A field reference such as `\y` is an ordinary field-id value. It can be stored or passed to another operation. Object merging uses `+`; when fields overlap, the right operand wins.

```
let defaults = {width: 80, height: 25};
let request = {width: 100};
let settings = defaults + request;
print settings\width;
```

Object mutation is visible through slot assignment. Mutable application state is passed to the operation that changes it; credited top-level scope is read-only, as described in Chapter 8.

SECTION 7

OPERATIONS

An operation is the ordinary unit of named computation. Positional parameters are declared in brackets and called in brackets.

```
op square [x] (  
  => x * x;  
)  
  
op sum_of_squares [a, b] (  
  => square[a] + square[b];  
)  
  
print sum_of_squares[3, 4];
```

The arrow => returns a value. It may be used early.

7.1 Shaped parameters

An operation may require a named object shape.

```
op magnitude {x, y} (  
  => x * x + y * y;  
)  
  
print magnitude {y: 4, x: 3};
```

Fields match by name rather than position. Shape mismatch is fatal. This is a contract, not an invitation for the callee to discover missing properties.

7.2 Infix vocabulary

Programmers may define readable infix operations.

```
op [x, y] dot [a, b] (  
  => x * a + y * b;  
)  
  
print [3, 4] dot [5, 6];
```

This facility permits domain notation without granting arbitrary parser extension. The definition and operand shapes remain visible.

7.3 Private operations and references

Prefix a helper with `private` when it implements a module but is not part of that module's credited surface: for example, `private op twice_internal [x] ( => x *`

2;). Operation references are explicit, validated values. There is no general pointer to arbitrary machine code.

SECTION 8

CREDITS, MODULES, AND AUTHORITY

Library operations do not arrive through a magical global namespace. A source file credits the modules whose vocabulary it uses.

```
| credits: <f/files> <f/bytes> <f/strings> <f/inspect>;
```

The credit is both dependency and declared authority. A program that does not credit filesystem or network operations should not acquire them accidentally. Tree shaking begins from the operations and data that remain reachable.

8.1 File-relative modules

Angle-bracket module paths can name standard modules or files relative to the current program.

```
| credits: <../lib/geometry.flx> <f/inspect>;
```

Public operations become available to the crediting source. Private operations stay within their defining source file.

8.2 Explicit scope

Operations do not silently capture top-level state. A definition that needs to read module data names it; the credited scope remains read-only.

```
| let configuration = {step: 1};  
  
| op advance [counter] (  
|   counter\value = counter\value + configuration\step;  
|   => counter;  
| ) has scope { configuration }  
  
| let counter = {value: 0};  
| counter = advance[counter];  
| print counter\value;
```

This rule prevents closure state from becoming invisible authority. The authorized read is visible at the definition, and an operation may not assign to it. Large applications keep their mutable world in an explicit capsule and pass it through the operations that alter it.

SECTION 9

BUGS AND THE WORLD

Freelang separates two kinds of failure that are frequently mixed together.

9.1 A bug ends the program

An out-of-bounds array read, impossible tag, broken heap header, missing field, or failed assertion is not a weather report. Correct code should not enter that state. The program therefore falls immediately.

```
op factorial_checked [n] (  
  assert(n >= 0);  
  let answer = 1;  
  let i = 2;  
  while (i <= n) (  
    answer = answer * i;  
    i = i + 1;  
  )  
  => answer;  
)  
  
print factorial_checked[5];
```

The purpose is not harshness for its own sake. It is to prevent poisoned state from travelling away from the violated invariant and becoming harder to attribute.

9.2 The world is data

Files can be absent. A peer can close. Permission can be denied. A process can be unavailable. These are possible states of the world, so an operation may declare has chaos and its caller must select a policy with `with chaos`.

```
credits: <f/process>;  
  
let status = program_run[  
  "/usr/bin/git",  
  ["git", "status", "--short"],  
  ["PATH=/usr/bin:/bin"]  
] with chaos {  
  NotFound(path) => 127  
  PermissionDenied(path) => 126  
  _ => 125  
};  
  
print status;
```


The executable path, argument vector, and entire child environment are explicit. No shell parses the arguments. A nonzero exit status is an ordinary result. Failure to start the selected executable is a named world state.

9.3 Handling, retrying, and bubbling

A handler arm can return a value, retry *N*, or bubble the state to its own caller. A catch-all policy is visible as `_`. Libraries should prefer named states and preserve information; applications decide when a specific state becomes a status code, message, retry, or exit.

This design removes two kinds of ambiguity. A programmer bug cannot be quietly treated as a routine operational result, and an expected world outcome cannot hide in an exception path the caller forgot to inspect.

SECTION 10

JOBS AND ISOLATION

Freelang jobs are processes. This is not an implementation detail hidden under an abstract task API. It is the concurrency contract. A fork block ends in `&`, returns a job handle, and creates another operating-system process with its own heap and collector. `wait` is the visible join.

The choice is intentionally severe. Threads make communication cheap by placing computations in one address space; they also make accidental sharing cheap. Freelang makes communication more expensive and gives isolation the default. A job cannot race another job by mutating the same array or object, because no heap object is shared between them.

10.1 The smallest fork and join

The following starts two jobs and waits for their returned values:

```
op sum_range [first, last] (  
  let total = 0;  
  let i = first;  
  while (i < last) (  
    total = total + i;  
    i = i + 1;  
  )  
  => total;  
)  
  
let left = ( ( => sum_range[0, 500]; ) )&  
let right = ( ( => sum_range[500, 1000]; ) )&  
  
let left_answer = wait left;  
let right_answer = wait right;  
print left_answer + right_answer;
```

The inner parentheses are the child block. The outer parentheses delimit the fork form. The trailing `&` is the visible process birth. Parent execution continues after each handle is created. `wait left` blocks only until `left` completes and then returns the value produced by the child's final `=>`. Waiting on the same handle again is idempotent. Waiting on a non-job value returns that value unchanged, which permits generic join code without inventing a second result wrapper.

The simple returned-value path is useful for an integer result. Named and structured communication uses FSABI.

10.2 FSABI in one sentence

FSABI is Freelang's filesystem application binary interface for jobs. Every job receives a private directory containing an inbox, an outbox, and lifecycle metadata.

Values cross the process boundary as self-describing files. The directory is both the calling convention and an inspectable record of the exchange.

The important claim is not that files are intrinsically safe. The claim is that a process boundary has no shared Freelang heap, the permitted directions are explicit, values are deep-copied through a bounded codec, and the state of the call can be examined with ordinary filesystem tools.

10.3 A complete value round trip

When a fork has named inputs or outputs, credit `f/fsabi`. Inputs appear before the child block. A child writes a named output with `value >name`; after joining, the parent reads it as a field of the job handle.

```
credits: <f/fsabi>;

let work = (
  <first = 20,
  <last = 22,
  (
    let answer = first + last;
    answer >sum;
    => 0;
  )
)&

wait work;
print work\sum;
```

There are two results in this example and they mean different things. `=> 0` is the child completion value observed by `wait`. The named `sum` is application data in the outbox. Keeping completion separate from outputs permits one job to report several named values without hiding them in an incidental tuple convention.

The source form lowers conceptually as follows:

1. allocate a job identity and create its capsule;
2. encode `first` and `last` into named inbox files;
3. mark the job running and start the same compiled child operation;
4. let the child decode its inputs into its own heap;
5. encode `sum` into the outbox and terminate with value zero;
6. let the parent join the process; and
7. decode `work\sum` into the parent's heap.

No parent pointer becomes meaningful in the child. The source names crossing the boundary are values, not addresses.

10.4 The capsule on disk

The runtime selects a platform temporary root and creates a job identifier as sixteen lowercase hexadecimal digits. Conceptually, the layout is:

```
<job-root>/job_<16-hex-digits>/
  inbox/
    params/      parent-to-child FSABI values
    files/       parent-to-child artifacts
  outbox/
    params/      child-to-parent FSABI values
    files/       child-to-parent artifacts
  meta/
    state        lifecycle state maintained by the runtime
```

On Unix the root is under `/tmp/freelang/jobs`. On Windows it is under the process's temporary directory. User code does not construct the root from an environment guess. The runtime exposes the selected root and job id to the standard FSABI module; the module constructs everything below that fixed point in ordinary readable Freelang.

Named parameter files end in `.flx`. For example, a binding named request occupies:

```
| .../inbox/params/request.flx
```

The direction rules are simple:

Box	Writer	Reader
Inbox	parent, before start	child
Outbox	child, while running	parent, after join
Meta	runtime	runtime and inspection tools

Writing into an inbox after the process starts, writing an outbox from the parent, or reading a result before completion violates the protocol. Such an operation falls. It is not converted into a timing-dependent empty value.

10.5 The lifecycle

A job has three conceptual phases.

NEW

The capsule exists. The parent may populate inbox parameters and artifacts. The child has not started.

RUNNING

Inputs are fixed. The child may read the inbox and write its outbox. The parent may retain the handle or wait, but may not observe partial results.

DONE

The process has terminated and the join has established completion. The parent may read stable outbox values and artifacts. Repeated waits return the cached result.

This lifecycle prevents a common ambiguity in asynchronous systems: whether a read means “no value,” “not yet,” “already consumed,” or “the producer failed.” Before completion, the read is invalid. After completion, absence is a named protocol failure rather than a guessed empty value.

10.6 The value format

Every FSABI parameter begins with the exact version line:

```
| freelang-fsabi 1
```

Exactly one value follows. Each value announces its type with an ASCII tag.

Tag	Type	Body
i	integer	signed decimal and newline
n	nil	no payload
s	string	UTF-8 byte length, bytes, newline
b	bytes	raw byte length, bytes, newline
a	array	element count, then encoded values
o	object	field count, then name/value pairs
@	operation	compiled operation name and newline

Strings and bytes are length-prefixed and never escaped. A newline, zero byte, or quotation mark inside a payload is data counted by the length, not syntax. Arrays recurse over values. Objects carry field names because in Freelang the field-name set is the shape.

An inspectable object file can therefore look like this:

```
| freelang-fsabi 1
| o 2
| s 4
| done
| i 0
| s 5
| title
| s 7
| ship it
```

The order is mechanical rather than literary, but no heap addresses or opaque compiler indexes are required to learn that the object contains `done: 0` and `title: "ship it"`.

10.7 Why the format is not JSON

JSON would provide familiarity but weaken two properties. First, JSON strings make escaping the central correctness problem; FSABI length-prefixes the bytes and has no escape character. Second, JSON has no native distinction between a Freelang string and byte array, and no representation for a callable operation reference or compiler-known object shape. A convention layered over JSON would be another format while retaining JSON's quoting surface.

JSON remains appropriate for untrusted interoperable documents. FSABI is a same-language job ABI with a narrower purpose. Its readable tags describe the Freelang values actually transferred.

10.8 Objects and operations cross by identity that can be checked

An object shape is not dynamically invented in the receiving process. The decoder reads the exact field names and searches the shapes compiled into the same binary. Hashes narrow the search, but every Unicode codepoint of every name is compared before a shape is selected. Duplicate fields, excessive counts, or a name set with no exact compiled shape fall.

An operation reference crosses by name, for example:

```
| freelang-fsabi 1  
| @ greet
```

The receiving process finds `greet` in its compiled operation table and refuses an unknown name. A table index would be smaller, but it would be meaningless to a person inspecting the file and dangerous if interpreted under the wrong binary. Same-binary reconstruction makes the name meaningful on both sides.

10.9 The trust boundary is the route, not the header

The version line identifies a format. It does not authenticate input. For that reason FSABI exposes two decode surfaces.

The public `fsabi_decode` operation accepts application bytes and reconstructs data only: integers, nil, strings, bytes, and arrays composed from those values. It refuses object and operation tags before consulting compiled shape or operation tables.

The inbox and outbox getters derive a path only from the runtime-selected job root and job identity. They enter a private same-binary decoder that may reconstruct an exact compiled object shape or operation reference. User source cannot call that private decoder, manufacture a `trusted = 1` flag, or turn a copied byte string into same-binary authority. Trust belongs to the runtime-resolved capsule route.

Both surfaces require the complete header, exactly one complete value, and no trailing bytes. A valid prefix cannot bless an appended payload.

10.10 Copy semantics and refused values

Transfer is always a deep copy. If two fields in the parent refer to one non-cyclic sub-value, the decoder may create two equal child values rather than preserving alias identity. Mutation in either process cannot propagate to the other.

Cycles are refused. The encoder tracks the current traversal path and falls if it encounters the same container again. A depth limit alone would merely turn a structural error into a mysterious size-dependent failure.

Some values have no portable job meaning. A job handle names a process in the parent and cannot cross as application data. A raw descriptor or socket number is not a portable capability: Unix fork may inherit a descriptor table, while a Windows worker re-executes and would receive an unrelated integer. Freelang therefore refuses values whose representation would look valid but mean something different on another target.

Malformed versions, truncated payloads, overrunning lengths, invalid UTF-8, unknown type tags, unknown operation names, impossible shapes, cycles, and uncrossable values all fall with named diagnostics. The boundary never returns a best-effort partial value.

10.11 Joining several jobs

Several handles may be joined explicitly:

```
wait first;
wait second;
print first\answer + second\answer;
```

The barrier form makes the continuation visually dependent on all handles:

```
weave first, second into (
  print first\answer + second\answer;
)
```

Here `weave ... into` is syntax for waiting on the listed handles in order and then executing the block in the parent process. It is not another worker, callback queue, or scheduler. A nested fork is permitted and receives its own capsule and process identity.

10.12 Unix fork and Windows re-execution

On native Unix targets the implementation can use `fork`: the child begins from copy-on-write process state and enters the compiled child operation. That physical inheritance does not license the language to expose shared mutable heap semantics. Cross-job application values still use the specified boundary.

Windows has no `fork`. The runtime starts the same executable with a private worker personality, job identity, and child entry name. It rebuilds the worker's runtime state and reads the same FSABI capsule. This is why an operation reference crosses by name and why inherited descriptor integers are not treated as portable values.

The source contract is identical: one binary, isolated process, explicit inputs, explicit join, named outputs. Platform mechanism is permitted to differ only beneath that contract.

10.13 Jobs, external programs, and sidecars are distinct

A *job* runs a compiled Freelang operation from the same executable and uses the same-binary FSABI boundary. `program_run` starts an arbitrary named OS executable synchronously with exact arguments and environment; that foreign program has no Freelang entry operation, heap, or FSABI identity. A *sidecar* is a compiler-selected, hash-pinned foreign capability process speaking a versioned protocol.

Combining these into one generic process handle would erase useful facts. The three mechanisms have different authority, value, lifecycle, and trust contracts, so the language keeps them separate.

10.14 What FSABI does and does not prove

FSABI provides process heap isolation, explicit directional transfer, a self-describing value representation, exact same-binary reconstruction rules, and an on-disk record that can be inspected during execution. These properties are implemented across current native targets and retained by round-trip and refusal tests.

It does not yet make every job hermetically replayable. A job may hold credited filesystem or network authority outside its capsule. Retention and cleanup are runtime policy rather than a permanent archive guarantee. The current capsule does not by itself prove that all external inputs were captured, and its version header is not a signature. Capability-bound replay manifests are a natural extension, but their absence is not disguised as a delivered property.

10.15 Cost, grain, and failure domain

Process isolation is appropriate for coarse work: independent searches, transformations over chunks, isolated request workers, tools that deserve a separate crash domain, and computations whose inputs and outputs are worth naming. A loop that starts a process for every integer has selected the wrong grain.

The process cost is not merely overhead; it buys a boundary the operating system understands. A child that violates an invariant can die without continuing in the parent's heap. The parent must still decide what job failure means for the application. Isolation contains corruption; it does not invent a correct recovery policy.

SECTION 11

FOREIGN CAPABILITY AND SIDECARS

Freelang has no general user-facing in-process FFI. Foreign code does not receive a pointer into the Freelang heap. When a platform capability requires foreign implementation, the normal mechanism is a *sidecar*: a separate, hash-pinned executable speaking a narrow versioned protocol.

11.1 The five properties

Every shipping sidecar is intended to be:

1. **Auditable:** exact bytes and source identity are recorded.
2. **Isolated:** it has its own address space and crash domain.
3. **Contract-tested:** an independent driver can test the wire.
4. **Version-locked:** the selected bytes are embedded at compile time.
5. **Lifecycle-bounded:** startup, protocol completion, and cleanup have explicit limits.

The GUI presenter and TLS cage are the principal shipping examples. On Linux, the GUI module speaks the X11 wire protocol directly, so no presenter sidecar is required there. The high-level program remains the same.

11.2 Chunky, never chatty

A process protocol must move meaningful batches. The presenter sends a frame and receives an event batch. The TLS cage moves streams of plaintext and ciphertext. A protocol that needs a round trip for each element in an inner loop is incorrectly shaped.

11.3 What the claim means

Freelang does not make foreign code safe. It makes the code separate, named, least-authorized, revalidated, and replaceable. Native memory corruption in a sidecar may destroy that sidecar. It does not thereby become a write into the Freelang heap.

This is a deliberately smaller claim than “secure by isolation.” The compiler, protocol validator, operating system mechanism, and permitted authority remain part of the trusted composition and are named as such.

SECTION 12

FILES, NETWORKS, AND WINDOWS

12.1 Text at boundaries

Freelang strings are codepoints; files and wires are bytes. Standard text file operations encode and decode UTF-8 explicitly in their implementation. Paths are checked scalar strings. Embedded NUL, surrogate values, and out-of-range codepoints are rejected rather than interpreted as truncated or platform- dependent names.

12.2 Network outcomes

The `f/net` module exposes TCP operations with named chaos states such as timeout, close, permission, and address failure. Applications match those states at the call. The educational server `examples/httpd-learn.flx` keeps the HTTP request bytes, route match, partial transport behavior, and failure policy visible in one file.

Its opening declaration is representative:

```
credits: <f/net> <f/process> <f/strings>
        <f/operators> <f/inspect> <f/bytes>;
```

The companion `examples/httpd.flx` then replaces educational local helpers with the standard `f/http` vocabulary. Reading the pair is an effective introduction to how readable source becomes a library contract.

12.3 A graphics loop

The GUI surface is a pixel buffer and a batch of events.

```
credits: <f/gui> <f/bytes> <f/inspect>;

let g = gui_open[480, 320, "hello"]
  with chaos { _ => 0 };
let buf = gui_buffer[480, 320];
let running = 1;

while (running == 1) (
  gui_fill[buf, gui_rgb[24, 24, 32]];
  gui_rect[buf, 480, 320, 40, 40, 80, 60,
    gui_rgb[240, 200, 60]];
  let events = gui_present[g, buf]
    with chaos { _ => [] };
  if (gui_has_close[events]) ( running = 0; )
)
gui_close[g];
```

Drawing is pure bounded pixel work. Presentation is the capability boundary. That separation makes renderer output testable without opening a window.

SECTION 13

MEMORY AND NATIVE ARTIFACTS

13.1 The value representation

At runtime, ordinary values occupy 64-bit words. Integers are immediate tagged values. Heap values use tagged pointers. Heap blocks have a type tag and a total size followed by a type-specific payload. Strings contain boxed codepoints; byte buffers contain raw bytes and are never scanned as pointers.

This representation is an implementation contract, not source syntax. Its importance to the programmer is the evidence it enables: the collector, each allocation site, and each backend agree on one layout.

13.2 Collection

The native runtime uses a non-moving mark-sweep collector with a coalescing free list. Compiled stack frames are scanned precisely by frame metadata. Heap blocks must tile the allocated region exactly. Broken metadata aborts loudly instead of inviting the collector to interpret arbitrary payload as pointers.

Three test modes make the collector work visible:

```
FREELANG_HEAP_BYTES=1M bash tests/run-all.sh
FREELANG_GC_STRESS=1 bash tests/run-all.sh
FREELANG_GC_PRECISE_ONLY=1 bash tests/run-all.sh
```

The first forces collection under small heaps. The second collects at every allocation. The third removes conservative gap scanning and therefore tests the precise-root contract directly.

13.3 Direct containers

The production compiler writes each native executable container itself:

Target	Artifact
Linux x86-64	Direct ELF64 executable
macOS x86-64	Direct Mach-O executable
macOS arm64	Direct Mach-O executable
Windows x86-64	Direct PE32+ executable
Web	WebAssembly module and bounded host agents

Assembly remains available as an oracle where applicable. Release evidence can record source revision, target, credited capability, sidecar hashes, container facts, and the gates that established the artifact’s claim.

SECTION 14

WEBASSEMBLY WITHOUT AMBIENT JAVASCRIPT

The Web target preserves the same architectural instinct in a different isolation unit. A Freelang module runs in a dedicated Worker and receives only named browser agents: a presenter for Canvas and input, local-artifact access, bounded derived-artifact storage, and a speaker for WebAudio.

Messages cross versioned structured-clone capsules. A small trusted adapter owns linear memory and validates each capsule. There is intentionally no generic “call arbitrary JavaScript” import. That would recreate the ambient FFI surface the sidecar design exists to avoid.

14.1 The Doom application

`examples/wasm-doom.flx` is a useful demonstration because it is not a toy drawing. JavaScript selects local WAD bytes. Freelang validates the archive, chooses a classic map, builds the texture and sprite bank, runs the world and combat state, renders every pixel, and emits the canonical speaker protocol. Canvas presentation and local file selection remain named Web authorities.

The same Doom source family also builds as native programs. The application has forced work in geometry, rasterization, sound, deterministic state, archive validation, GUI input, Web capsules, and native release evidence. In Freelang’s development discipline, such an application is not merely a demo; it is evidence that the language boundary survives an unfamiliar workload.

SECTION 15

A TOUR OF THE EXAMPLE PROGRAMS

The repository examples are arranged less as a graduated exercise book than as a shelf of working instruments. The Builder's Kit supplies the sources and suggested modifications. The following reading order is effective.

15.1 Small transformations

examples/hello.flx

The irreducible program: one observable statement.

examples/rot13.flx

Strings, codepoints, iteration, and a complete small transformation.

examples/rainsum.flx

Readable hashing and arithmetic suitable for comparison with a native kernel or external fixture.

15.2 Auditable system tools

examples/git-status.flx

An absolute executable, explicit argument array, explicit environment, inherited standard streams, and named launch outcomes. It is the direct answer to an opaque shell call.

examples/auditable-tool.flx

Shaped data plus a forked job and visible result transfer.

examples/hexview.flx

A complete terminal hex viewer with bounded byte reads, input handling, frames, paging, and goto state.

examples/flx-curl.flx

HTTPS through the TLS cage without linking a TLS library into the Freelang program.

15.3 Servers and interfaces

examples/httpd-learn.flx

A one-file HTTP/1.1 server that exposes bytes and world outcomes.

examples/httpd.flx

The same problem expressed through the standard HTTP module.

examples/gui-suite.flx

Cross-platform pixel and event behavior through the presenter contract.

examples/snake.flx

A complete terminal game. The snake, food, direction, clock-derived random state, collision logic, and frame all remain readable in one source file.

15.4 Substantial applications

examples/jpeg-check.flx and examples/jpeg-bench.flx

A checked image codec workload, useful for examining the boundary between readable algorithms and proved native kernels.

examples/wasm-gallery/app.flx

A Web application using the bounded presenter and artifact-agent model.

examples/wasm-doom.flx

A local-artifact Doom application sharing the native engine, including the laser blaster, exploration shield, and FPV drone additions in the current application line.

No example should be treated as sacred. Change a constant, remove a credit, force a chaos arm, reduce the heap, inspect the assembly, and compare the result. The examples are working arguments about the language.

SECTION 16

A DISCIPLINED BUILDING METHOD

16.1 Begin with the world

Before writing the success path, enumerate what the outside world may do. A file might be absent; a peer might close; a sidecar might not start. Decide which layer owns the policy. Libraries preserve the state. Applications turn it into retry, alternate work, a diagnostic, or termination.

16.2 Name authority

Credit only the modules the program uses. Pass explicit state rather than inventing ambient global machinery. When starting another process, name its absolute path, arguments, and environment. When foreign capability is needed, select an existing protocol or design a bounded sidecar; do not widen the language with a convenient in-process call.

16.3 Keep invariants fatal

Use `assert` and `expect_eq` to keep impossible states close to the code that created them. Do not convert a broken bound or type into a sentinel and continue. A sentinel is appropriate only when it is an ordinary state of the modeled domain.

16.4 Build from applications, then retain proofs

Freelang's roadmap uses applications to discover missing language features. A proposed feature must be justified by concrete application pressure, land independently, retain tests, and avoid widening an unproved boundary. After a green stage, code, tests, commit evidence, and the progress journal are reconciled. Experimental dead ends remain in lab notes rather than being edited out of history.

16.5 Inspect what was built

For a serious artifact, inspect more than its stdout:

- confirm the exact source revision and target;
- inspect imports and container structure;
- verify signatures and hashes where distributed;
- examine credited modules and embedded sidecars;
- run small-heap and stress collection tests after runtime work; and
- retain complete suite logs and state only what their evidence proves.

SECTION 17

THE POSITION IN FULL

Freelang is a small direct-native language for human and machine authors. Its central problem is not the speed at which source can be generated, but whether that generation can be amplified without multiplying old silent failures. The language therefore prefers explicit world states, fatal invariants, compiler-known value shapes, declared authority, process-isolated concurrency, and finite native boundaries.

17.1 The language contract

Ordinary source consists of values, lexical bindings, closed named operations, visible control, credited modules, and explicit boundary operations. Strings are codepoints and wires are bytes. Arrays are immutable positional values; objects have named compiler-known shapes. Dynamic access remains visible as dynamic access. Operations do not silently capture mutable outer state.

External uncertainty is returned as declared chaos data. Bugs fall. Jobs are processes joined by `wait`; FSABI transfers deep-copied values through inspectable directional capsules. Foreign stateful capability belongs behind a separately identified process and bounded protocol. Direct native backends emit the executable container and only the runtime facilities the reachable program requires.

17.2 The engineering contract

Every widening of the language is expected to retain evidence appropriate to its claim. Source semantics need positive and expected-failure tests. Native helpers need cross-target probes and calling-convention audits. Allocation changes need small-heap, stress, and precise-root exercises. A sidecar needs a wire specification, independent driver, negative protocol cases, exact binary identity, and bounded lifecycle. A release needs source identity, hashes, signatures where supported, and inspectable build evidence.

The implementation may optimize a readable operation with a native kernel, but the readable behavior remains the contract and the kernel must be proved against it. The project does not acquire a general unsafe programming mode merely because one optimized loop was justified.

17.3 The admitted limits

Freelang does not presently claim formal verification, complete memory safety, constant-time cryptography, immunity from compiler bugs, protocol correctness, supply-chain perfection, or a sandbox around every credited operation. A non-moving collector and checks reduce particular hazards; they do not prove all memory behavior. Process boundaries contain address-space corruption; they do not make the remote process benevolent. Signatures identify published bytes; they do not make those bytes

correct.

The WebAssembly target is a reduced-authority dialect. It can preserve pure computation and named browser presentation agents, but it cannot honestly provide native fork-based jobs. The distinction is recorded rather than hidden behind an emulation that would alter the meaning of concurrency.

17.4 Why the guide remains provisional

The word *canonical* means that this volume states the whole design position needed to understand the language; it does not mean the pre-release language is frozen. Source, tests, compiler artifacts, and bounded evidence remain the final arbiter when implementation and print temporarily differ. Substantive changes should produce a revised edition rather than accumulating folklore outside the book.

Freelang's claim to importance will not be established by the novelty of its origin or the confidence of this manual. It will be established if unfamiliar people and models build unfamiliar programs, make fewer silent mistakes, and leave evidence that another party can inspect.

SECTION A

SYNTAX SUMMARY

Form	Meaning
<code>let x = value;</code>	Introduce a lexical binding.
<code>if (test) (...)</code>	Conditional block.
<code>while (test) (...)</code>	Repetition while nonzero.
<code>forall x in a (...)</code>	Traverse an array; may construct an array.
<code>op f [a,b] (...)</code>	Define and call a positional operation.
<code>op f {a,b} (...)</code>	Define a shaped object operation.
<code>=> value;</code>	Return or emit a value.
<code>a\0</code>	Literal positional slot.
<code>a\\i</code>	Dynamic slot.
<code>a@i</code>	Coequal dynamic slot spelling.
<code>obj\name</code>	Literal object field.
<code>\name</code>	Field identifier as a value.
<code>credits: <f/name>;</code>	Credit module vocabulary and authority.
<code>has scope { world }</code>	Authorize named top-level scope.
<code>call[...] with chaos {...}</code>	Handle declared world states.
<code>((=> work[];))&</code>	Start an isolated same-binary job.
<code>(<x = v, (x >out; => 0;))&</code>	Start a job with FSABI input/output.
<code>wait job</code>	Join a job and obtain its completion value.
<code>job\out</code>	Decode a named outbox value after join.
<code>weave a,b into (...)</code>	Join several jobs, then run a parent block.
<code>assert(test);</code>	End the program if an invariant is false.

SECTION B

COMPILER COMMAND SUMMARY

```
# Compile and run from a source checkout
./flx.sh program.flx arg1 arg2

# Compile only for the host
./flx.sh --build-only program.flx

# Direct native targets with the source compiler
node freelang.js in.flx out.bin --target=linux --emit=bin
node freelang.js in.flx out.bin --target=darwin --emit=bin
node freelang.js in.flx out.bin --target=darwin-arm64 --emit=bin
node freelang.js in.flx out.exe --target=windows --emit=bin

# Use a standalone release compiler
./freelang in.flx out.bin --target=linux

# Retain readable x86-64 assembly
./freelang in.flx out.s --target=linux --emit=asm
```

Build and compiler options precede the source when using the wrapper. Arguments after the source belong to the compiled program. The command summary records the stable ordinary compilation forms; experimental and diagnostic flags are not part of the language contract.

SECTION C

GLOSSARY

Artifact

An uninterpreted file transferred or produced at a boundary, or a complete emitted executable. Context distinguishes the two uses.

Authority

Permission to affect or observe something outside pure local computation, such as files, networks, processes, windows, or audio devices.

Capsule

The complete directory associated with one job: inbox, outbox, and runtime metadata. It is the inspectable unit of an FSABI exchange.

Chaos

A declared set of possible world states returned as data. Chaos is not an exception mechanism and does not represent programmer bugs.

Credit

A source declaration that admits a module's public vocabulary and records its capability class for reachability and inspection.

Fall

Immediate termination caused by a violated invariant, invalid program operation, corrupt representation, or refused boundary value.

FSABI

The filesystem application binary interface used by same-binary Freelang jobs for named, deep-copied, inspectable value transfer.

Intrinsic

A compiler-owned, manifest-listed native operation. Intrinsic form a finite implementation boundary; they are not a general FFI.

Job

An isolated same-binary Freelang operating-system process created by a fork block and joined with `wait`.

Operation

A closed named unit of Freelang computation, introduced with `op`.

Shape

The compiler-known field-name set and layout of an object or shaped operation parameter.

Sidecar

A separate hash-pinned foreign executable that owns one capability and communicates over a narrow versioned protocol.

World state

An expected external outcome such as absence, denial, timeout, close, or resource exhaustion. It belongs in a chaos domain rather than a fatal path.

BUILDER'S KIT NOTICE

The Freelang Builder's Kit is the companion working set for this guide. It is intended for readers who want the example sources, compiler materials, exercises, build notes, and inspectable artifacts together rather than as isolated excerpts.

Suggested first projects include:

1. alter `examples/rot13.flx` to implement another reversible alphabet transformation;
2. add a named route to `examples/httpd-learn.flx`, then make the corresponding change through `f/http`;
3. add one visible state to `examples/snake.flx` without introducing hidden mutable scope;
4. inspect the native imports and release evidence for the result; and
5. build one intentionally failing program for every invariant relied on by the successful version.

THE FREELANG BUILDER'S KIT. Companion source, exercises, compiler materials, and working programs are available separately. For ordering information write freelang@dosaygo.com or compiler@freelang.dev.
--